

World of Warships Monetization Case Study

Project Overview

This project serves as an end-to-end data analytics case study exploring the monetization strategies and in-game economy of World of Warships. By programmatically extracting ship attributes and player performance statistics directly from the Wargaming.net Public API, this study investigates the behavioral and economic differences between players using standard Tech Tree ships versus purchasable Premium ships. Using a custom data pipeline built with Python, Google BigQuery (SQL), and Power BI, I processed and analyzed engagement metrics—such as usage rates, win rates, and base experience generation—to uncover actionable trends in player purchasing patterns. The resulting insights are designed to help Wargaming's product and balancing teams optimize future Premium ship releases to maximize both long-term player retention and revenue.

Phase 1: Ask

Business Task: To analyze the usage rates, in-game performance, and player engagement levels of Premium ships compared to Tech Tree ships. By identifying which tiers, classes, and specific types of Premium ships drive the highest engagement and ownership, I will provide data-driven recommendations on what types of Premium ships Wargaming should focus on releasing next to maximize monetization and player retention.

Key Stakeholders:

- **Wargaming's Monetization and Product Teams:** They need to understand player purchasing patterns and engagement to decide how to price ships and what types of bundles or Premium ships to sell for Doubloons (the primary cash currency).
- **Game Balancers / Development Team:** They need to ensure that the Premium ships being released to drive revenue are not negatively impacting the overall game balance (e.g., ships that are heavily played but have disproportionately high or low win rates).

Technical Toolkit

- **Python:** Fetch raw data from Wargaming API.
- **Excel/Sheets:** Cleaning and summarization.
- **SQL:** Processing and combining into one table.
- **Power BI:** Advanced and interactive visualizations.

Repository Structure

- **01_raw_data/:** Untouched JSON/CSV output fetched from Wargaming API.
- **02_processed_data/:** CSV files of raw data cleaned and summarized, ready for final reporting.
- **03_scripts_notebooks/:** Python script to collect data and SQL script to process data
- **04_visualizations/:** Final charts in PDF and PNG formats, plus the JSON Power BI theme I made by scraping World of Warships' official branding colors.
- **05_reports/:** Executive summary and final project PDF.

Phase 2 & 3: Prepare & Process (Data & Methodology)

Data Sources: The data for this case study is extracted directly from the **Wargaming.net Public API**, a first-party developer service providing live access to game content and player statistics. Specifically, this analysis utilizes two main API endpoints:

- **Encyclopedia API** (`/wows/encyclopedia/ships/`): Used to extract static ship attributes, such as tier, nation, ship class, and the critical `is_premium` status.
- **Warships API** (`/wows/ships/stats/`): Used to extract historical player performance data on specific ships, including usage rates, win/loss ratios, and base experience generated.
- **Players API** (`/wows/account/list/`): Due to limitations of the Warships API, I need to construct a representative sample without brute-forcing random player IDs. To accomplish this, the data collection script programmatically queries this endpoint using a predefined list of 10 common username prefixes. This extracts 1,000 valid `account_ids`, which are then used to fetch individual ship performance records.

Data Integrity:

- **Credibility (ROCCC):** I am pulling data directly from Wargaming's own servers, so the data is first-party, reliable, original, and current. It is also unbiased because it reflects the exact database used to run the game.
- **Technical Integrity:**
 - Checking the API response header for `status: ok`, which confirms the request was processed successfully without returning an `error` dictionary.
 - Managing API rate limits (standalone applications are limited to 10 requests per second) to ensure data pull is not truncated or missing record.
 - Checking final output (e.g., CSV files) for missing or null values—such as test ships or newly released ships that might lack complete public statistics.

Data Privacy & Licensing:

- **Licensing:** I am operating under Wargaming's limited, non-exclusive, revocable license to use their API data for this publicly available application/analysis.
- **Copyright Notice:** Wargaming's policy requires the inclusion of the notice, "© Wargaming.net. All rights reserved".
- **Attribution:** The source of the data is the [official World of Warships website](#) and this project is not affiliated with or endorsed by Wargaming.
- **Privacy:** No personal passwords were collected and only the data necessary for the analysis was collected.

How to Reproduce the Data Collection: To reproduce the API extraction process and run the Python scripts locally, you will need to set up your environment and obtain your own API credentials. Follow these steps:

1. **Install Required Python Packages:** The data collection scripts utilize external libraries that must be installed via your computer's terminal or command prompt (do not run this command inside the Python script). Open your terminal and run the following command: `pip install requests pandas python-dotenv`
2. **Obtain a Wargaming Application ID:** Access to the Wargaming.net API requires a personal Application ID. Register for a free developer account at the [Wargaming Developer Room](#) and create a "Standalone" application to generate your key.

3. Set Up Your Environment Variable: To keep your Application ID secure and comply with Wargaming's Terms of Use, the Python script uses the `python-dotenv` package to load the key as a local environment variable.

- In the root directory of your cloned repository, create a plain text file named exactly `.env`.
- Add your Application ID to the file in this exact format:
`WG_APP_ID=your_application_id_here`
- **Important:** Ensure your `.env` file is listed in your `.gitignore` file so your secret key is never pushed to your public repository!

Data Volume & Sampling Note: The default configuration of the data collections script uses 10 common username prefixes to query the `/wows/account/list/` endpoint, extracting 983 valid `account_ids`. Of those valid `account_ids`, 527 returned stats. This default sample yields 9,810 individual ship performance records. While this is sufficient for the scope of this case study, it suggests a high volume of either:

- **Hidden Profiles:** Wargaming allows players to hide their statistics. The API explicitly tracks a `hidden_profile` status, and if a player has their profile set to private, the API will not return their ship performance data.
- **Zero PvP Battles:** The Python script was specifically designed to filter out ships with 0 Random Battles (`pvp`) played. Many accounts may only play Co-op battles (which the game actively incentivizes with discounted service costs) or may have simply created an account without ever playing a match.

Data Cleaning & Preparation Steps: Because the data was extracted programmatically via API, it lacked traditional errors like null values or misaligned columns. Using Google Sheets/Excel, the following cleaning tasks were completed:

- **wows_ship_encyclopedia.csv:**
 1. Removed empty rows at the end of the data.
 2. Ensured accurate data formatting.
 3. Sorting `name` alphabetically, I found some ship names inside brackets, which indicates the player did not own the ship, but rather rented it. Because the business task is specifically focused on comparing standard Tech Tree ships to purchasable Premium ships, these temporary rentals were removed.
 4. Two ships were listed twice ("Schlieffen" & "Vrijheid") with different ship IDs. This was determined to be caused by developers releasing beta versions of the ships to be tested, changing the `ship_id` to release the final version of the ship. I resolved this by searching `wows_player_ship_stats.csv` for the IDs. Only 1 of the "Schlieffen" ships were played, so I deleted the others as they would have been removed when I JOIN the tables later.
 5. Added `event` column to indicate ships purchased during a special event or ones offered with special variants.
- **wows_player_ship_stats.csv:**
 1. Removed empty rows at the end of the data.
 2. Ensured accurate data formatting.

Phase 4: Analyze (Summary of Analysis)

To answer the core business questions, I utilized Google BigQuery to build a centralized `master_ship_summary` table, joining the static ship attributes with player performance records. I then wrote targeted SQL queries to aggregate engagement and economic metrics.

How to Reproduce the Data Analysis: You will need to replace `[your-project-id]` placeholders with your own personal BigQuery project ID.

SQL Highlight: CTEs for Apples-to-Apples Balancing To accurately compare Premium ships against standard Tech Tree ships, I used Common Table Expressions (CTEs) to isolate the averages. By joining the two temporary tables on shared traits (`tier`, `nation`, and `type`), the query output places a Premium ship's win rate, damage, and XP directly next to the average of its exact Tech Tree counterparts.

```
WITH premium_stats AS (
  SELECT
    ship_name,
    tier,
    nation,
    type,
    SAFE_DIVIDE(SUM(total_wins), SUM(total_battles)) AS prem_win_rate,
    SAFE_DIVIDE(SUM(total_damage_dealt), SUM(total_battles)) AS prem_avg_damage,
    SAFE_DIVIDE(SUM(total_xp), SUM(total_battles)) AS prem_avg_xp
  FROM `[your-project-id].wows_case_study.master_ship_summary`
  WHERE is_premium = TRUE
  GROUP BY ship_name, tier, nation, type
),

tech_tree_stats AS (
  SELECT
    tier,
    nation,
    type,
    SAFE_DIVIDE(SUM(total_wins), SUM(total_battles)) AS tt_win_rate,
    SAFE_DIVIDE(SUM(total_damage_dealt), SUM(total_battles)) AS tt_avg_damage,
    SAFE_DIVIDE(SUM(total_xp), SUM(total_battles)) AS tt_avg_xp
  FROM `[your-project-id].wows_case_study.master_ship_summary`
  WHERE is_premium = FALSE AND event = 'Standard'
  GROUP BY tier, nation, type
)

SELECT
  p.ship_name,
  p.tier,
  p.nation,
  p.type,
  p.prem_win_rate,
  t.tt_win_rate,
  p.prem_avg_damage,
  t.tt_avg_damage,
  p.prem_avg_xp,
  t.tt_avg_xp
FROM premium_stats p
INNER JOIN tech_tree_stats t
  ON p.tier = t.tier
  AND p.nation = t.nation
  AND p.type = t.type
ORDER BY
```

```
p.tier DESC,  
p.nation,  
p.type,  
p.ship_name;
```

(Note: The full suite of queries used for event engagement and ownership proxies can be found in the *03_scripts_notebooks* folder).



Phase 5: Share (Visualizations & Key Findings)

I created a custom Power BI JSON theme by scraping World of Warships' official branding colors to ensure an authentic and visually cohesive visualizations, which is included in the *04_visualizations/* directory.

1. Event Engagement Chart Event Engagement **Key Findings: Event Engagement**

- **Black Friday Dominates:** "Black Edition" ships are by far the most popular special-event vessels, driving **2,657 battles** in the sample. This is more than double the engagement of the next closest category.
- **Anime/Collaboration Success:** The "Arpeggio" collaboration ships are the second most popular event category (**1,204 battles88), proving that targeted franchise crossovers successfully drive player engagement.
- **Underperforming Categories:** "Golden Edition" and "Special Distribution Variants" show extremely low usage, indicating these types of sales events may not effectively drive long-term player retention.

2. In-Game Performance (Premium vs. Tech Tree) In-Game Performance **Key Findings: Balancing & Economy**

- **Economic Superiority:** The KPI cards confirm that Premium ships consistently yield higher Average Base XP than their exact Tech Tree counterparts. This validates to players that purchasing Premium ships is a highly effective way to "grind" through the game's economy.
- **Balancing Outliers:** The scatter plot visualizes the balance spread. While most Premium ships cluster safely around the Tech Tree averages, stakeholders can use this view to easily identify specific Premium outliers that are either severely over-performing (high win rate/high damage) or under-performing.

3. Ownership Proxy & Tech Tree Usage Ownership Proxy & Tech Tree Usage **Key Findings: Usage Comparisons**

- **Tech Tree Ships Dwarf Premium Usage:** By synchronizing the X-axis of both charts, it is visually apparent that free, standard Tech Tree ships are played vastly more than purchased Premium ships.
- **The Top Performers:** The most popular Premium ship is the Tier 7 **Atlanta** (2,264 battles). However, the most popular Tech Tree ship is the Tier 10 **Shimakaze** (6,290 battles), which boasts nearly triple the engagement.
- **Tier 10 Drives the Game:** The Tech Tree usage chart reveals that Tier 10 ships (like the *Shimakaze* and *Montana*) drive the absolute highest engagement in the game, meaning players are spending the majority of their time at the "end-game" rather than lingering in the mid-tiers where most Premium ships exist.



Phase 6: Act (Strategic Recommendations & Insights)

High-Level Insights & Recommendations Based on the analysis of ship usage, in-game performance, and event engagement, here are my top recommendations for Wargaming's monetization and product teams:

1. **Capitalize on Tier 10 Engagement:** The data overwhelmingly proves that Tier 10 Tech Tree ships (like *Shimakaze* and *Montana*) drive the highest engagement in the game, dwarfing Premium ship usage.
 - **Recommendation:** Wargaming should shift its monetization focus toward the "end-game" by releasing more Tier 10 Special ships or pushing high-tier permanent camouflages, capturing the audience where they spend the majority of their time.
2. **Expand the "Black Edition" Model:** "Black Friday" variant ships see more than double the engagement of any other special event category.
 - **Recommendation:** The product team should replicate this successful formula of offering highly desired Premium ships at a discounted rate with exclusive black skins for other major seasonal events to drive consistent revenue spikes.
3. **Proactive Balance Monitoring:** The KPI cards confirm Premium ships offer a distinct economic (Base XP) advantage, but their win rates must remain fair.
 - **Recommendation:** The balancing team should actively monitor the Power BI scatter plot to easily identify Premium ships that are underperforming compared to their Tech Tree counterparts. Buffing these outliers will improve player satisfaction and drive future sales.

Further Exploration & Additional Deliverables To expand upon these findings, a helpful next step would be to pull additional data to analyze advanced game economies. Future analysis could include:

- Extracting data regarding the **Research Bureau** to see how heavily veteran players are resetting their Tech Tree lines to earn Research Points.
- Analyzing the ownership rates of Premium ships purchased with real-world currency (Doubloons) versus those purchased with "freemium" earned in-game resources like Coal or Steel.



Connect with Me

- **Portfolio:** austinbynum.com
- **GitHub:** github.com/austinbynum
- **LinkedIn:** linkedin.com/in/austin-bynum-323228315
- **Kaggle:** kaggle.com/austinbynumahs10